Puissance 4

Technical Report

Achagui Aymen

ENSEEIHT — Computer Engineering Student

Stack	Java 17, Spring Boot 3, PostgreSQL
DevOps	Docker, Kubernetes (minikube/EKS), GitHub Actions
AI	Gemini 3.5 Flash (PR Reviewer prototype)

Contents

Project Overview

Puissance 4 is a cloud-native real-time multiplayer Connect Four application built as a full-stack engineering project. The goal was not only to implement the game logic but to architect, containerize, and deploy a production-grade backend with a complete CI/CD pipeline.

The application exposes a REST API for authentication, game management, and rankings, complemented by a WebSocket layer for real-time gameplay. The entire stack is containerized with Docker and deployed on Kubernetes, with an AI-powered pull request reviewer integrated into the development workflow.

Architecture Overview

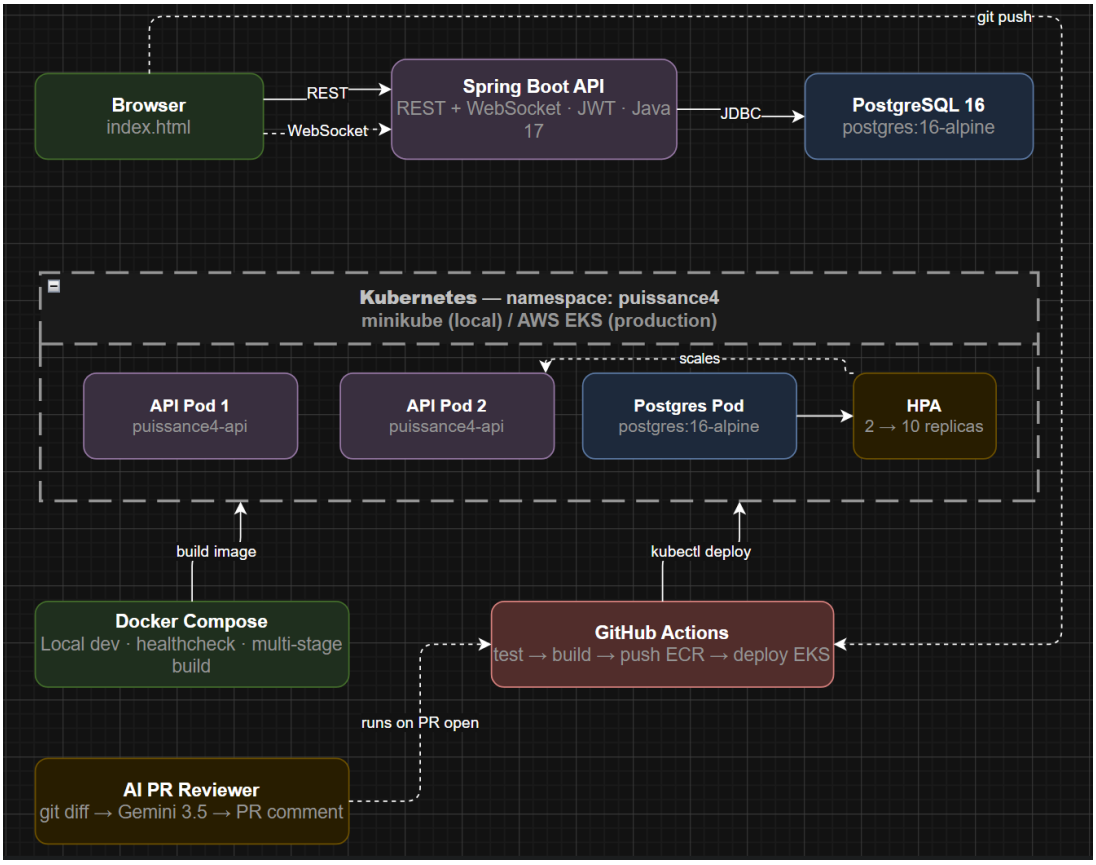


Figure 1: Application architecture

Application Features

Authentication

JWT-based authentication. Users register and receive a signed token, which is validated on every subsequent REST request and WebSocket message.

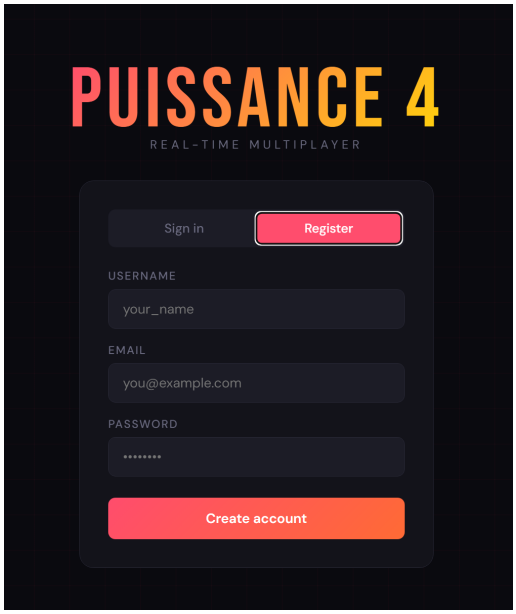
Game Logic

The board is stored as a serialized string: `0000000,0000000,...` — 6 rows of 7 columns. Token placement applies gravity (scanning upward from the bottom row). Win detection checks all four directions after every move: horizontal, vertical, diagonal, and anti-diagonal.

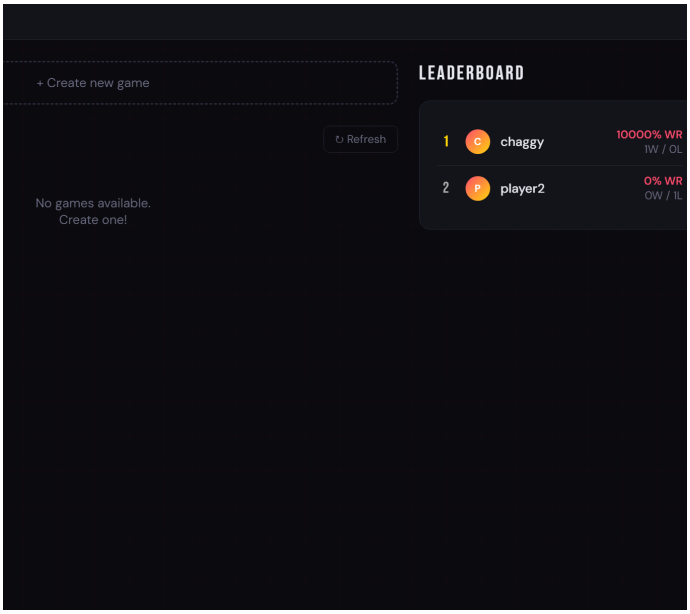
Real-Time Communication

WebSocket with STOMP protocol. Moves, chat messages, and game events are broadcast to both players via `/topic/game/{gameId}` without polling.

Frontend



(a) Login / Register



(b) Game Lobby

Figure 2: Frontend — authentication and lobby

Containerization

Multi-Stage Dockerfile

A multi-stage build keeps the final image minimal and secure. The build stage compiles the application with Maven; the runtime stage uses a minimal Alpine JRE image and runs as a non-root user.

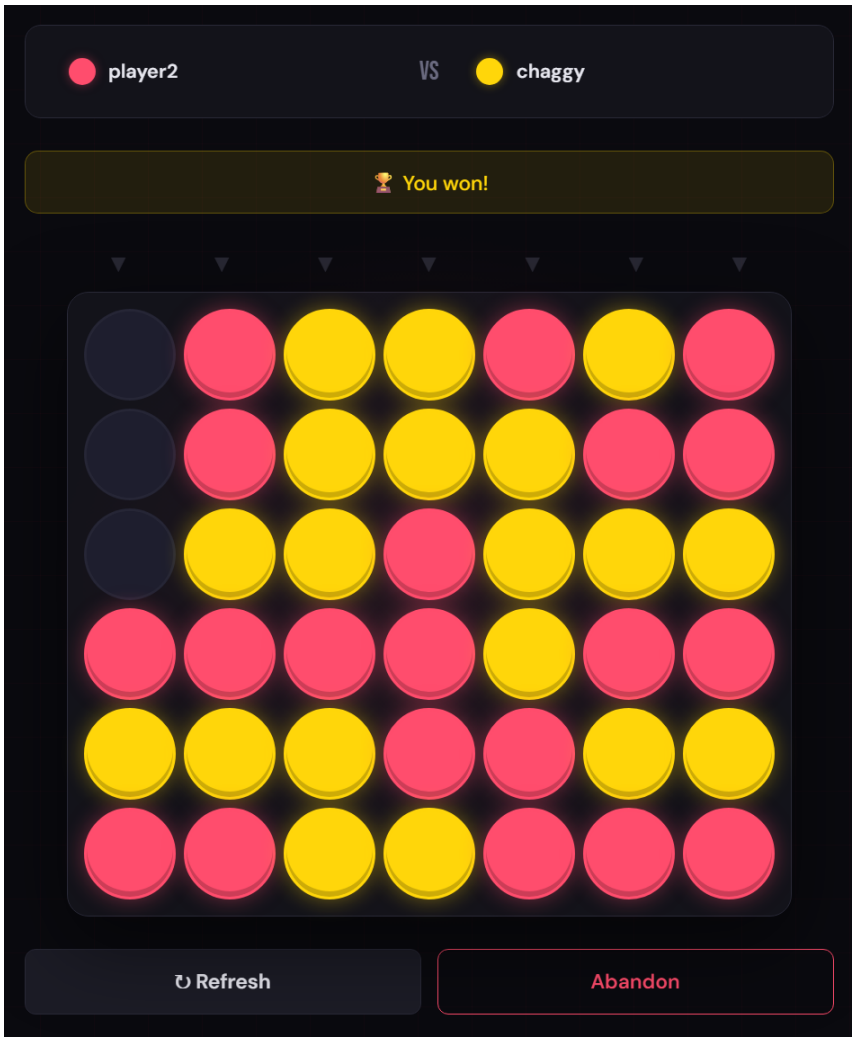


Figure 3: Game board — real-time play with turn indicators

```dockerfile
# Stage 1: Build
FROM maven:3.8.7-eclipse-temurin-17 AS builder
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline -q
COPY src ./src
RUN mvn package -DskipTests -q

# Stage 2: Runtime (minimal image)
FROM eclipse-temurin:17-jre-alpine
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
WORKDIR /app
COPY --from=builder /app/target/*.jar app.jar
USER appuser          # non-root for security
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Listing 1: Multi-stage Dockerfile

Docker Compose

Local development stack with a PostgreSQL healthcheck — the API only starts once the database is fully ready.

```yaml
services:
  db:
    image: postgres:16-alpine
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]

  api:
    build: .
    depends_on:
      db:
        condition: service_healthy    # wait for DB before
           starting
```

Listing 2: docker-compose.yml (simplified)

Kubernetes Deployment

The application is deployed on Kubernetes (locally via minikube, production-ready for AWS EKS). The deployment uses a rolling update strategy to ensure zero downtime.

Deployment Strategy

```yaml
spec:
  replicas: 2
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1         # allow 1 extra pod during update
      maxUnavailable: 0   # never kill old pod before new one is
         ready
  template:
    spec:
      containers:
        - name: puissance4-api
          livenessProbe:
            httpGet:
              path: /api/auth/health
              port: 8080
          readinessProbe:
            httpGet:
              path: /api/auth/health
              port: 8080
```

Listing 3: deployment.yaml — rolling update configuration

Horizontal Pod Autoscaler

The HPA automatically scales the deployment from 2 to 10 replicas based on CPU and memory utilization.

```yaml
spec:
  scaleTargetRef:
    kind: Deployment
    name: puissance4-api
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          averageUtilization: 60
    - type: Resource
      resource:
        name: memory
        target:
          averageUtilization: 70
```

Listing 4: hpa.yaml

```
chaggyneutron@CHAGGYYYY:~/puissance4$ kubectl get pods -n puissance4
NAME                                READY   STATUS    RESTARTS   AGE
postgres-5cf8d69b86-dfttw           1/1     Running   0          23h
puissance4-api-86948dc664-68phk     1/1     Running   0          137m
puissance4-api-86948dc664-fjhwq     1/1     Running   0          135m
```

Figure 4: Kubernetes pods running locally on minikube

CI/CD Pipeline

The GitHub Actions pipeline runs on every push to `main` and every pull request. It consists of three jobs: test, build & push to ECR, and deploy to EKS.

```yaml
jobs:
  test:                              # runs on every push and PR
    steps:
      - name: Run tests
        run: ./mvnw test

  build-push:                        # only on main branch
    needs: test
    steps:
      - name: Build and push to ECR
        run: |
          docker build -t $REGISTRY/$ECR_REPO:${{ github.sha }} .
          docker push $REGISTRY/$ECR_REPO:${{ github.sha }}

  deploy:                            # only after build-push succeeds
    needs: build-push
    steps:
      - name: Rolling update on EKS
        run: |
          kubectl set image deployment/puissance4-api \
            api=$IMAGE -n puissance4
      - name: Verify rollout
        run: kubectl rollout status deployment/puissance4-api --
           timeout=5m
```

Listing 5: ci-cd.yml — pipeline structure

Each image is tagged with the Git commit SHA for full traceability. Rolling back to a previous version is as simple as:

`kubectl set image deployment/puissance4-api api=registry/repo:<previous-sha>`

AI-Powered PR Reviewer

Motivation

One of the key DevOps challenges is maintaining code quality at scale. This prototype automates pull request reviews by sending the git diff to an LLM and posting the feedback as a GitHub comment — directly in the developer's workflow.

Architecture

`Open PR` → `Fetch diff (GitHub API)` → `Send to Gemini` → `Post comment on PR`

Implementation

```python
def get_pr_diff(pr_number):
    """Fetch the raw diff of a PR via GitHub API."""
    url = f"https://api.github.com/repos/{REPO}/pulls/{pr_number}"
    headers = {
        "Authorization": f"token {GITHUB_TOKEN}",
        "Accept": "application/vnd.github.v3.diff"
    }
    return requests.get(url, headers=headers).text

def review_with_gemini(diff):
    """Send diff to Gemini and get a structured code review."""
    prompt = f"""You are a senior code reviewer. Analyze this git diff and provide:
1. Security issues
2. Bugs
3. Bad practices
4. Summary of changes

Git diff:
{diff}
"""
    response = requests.post(
        f"https://generativelanguage.googleapis.com/v1beta/models/"
        f"gemini-3.5-flash:generateContent?key={GEMINI_API_KEY}",
        headers={"Content-Type": "application/json"},
        json={"contents": [{"parts": [{"text": prompt}]}]}
    )
    data = response.json()
    if "error" in data:
        return f"Gemini error: {data['error']['message']}"
    return data["candidates"][0]["content"]["parts"][0]["text"]

def post_pr_comment(pr_number, comment):
    """Post the review as a comment on the GitHub PR."""
    url = f"https://api.github.com/repos/{REPO}/issues/{pr_number}/comments"
    body = f"## AI Code Review\n\n{comment}\n\n*Powered by Gemini 3.5 Flash*"
    response = requests.post(url,
        headers={"Authorization": f"token {GITHUB_TOKEN}"},
        json={"body": body})
    return response.status_code == 201
```

Listing 6: ai_reviewer.py — core logic

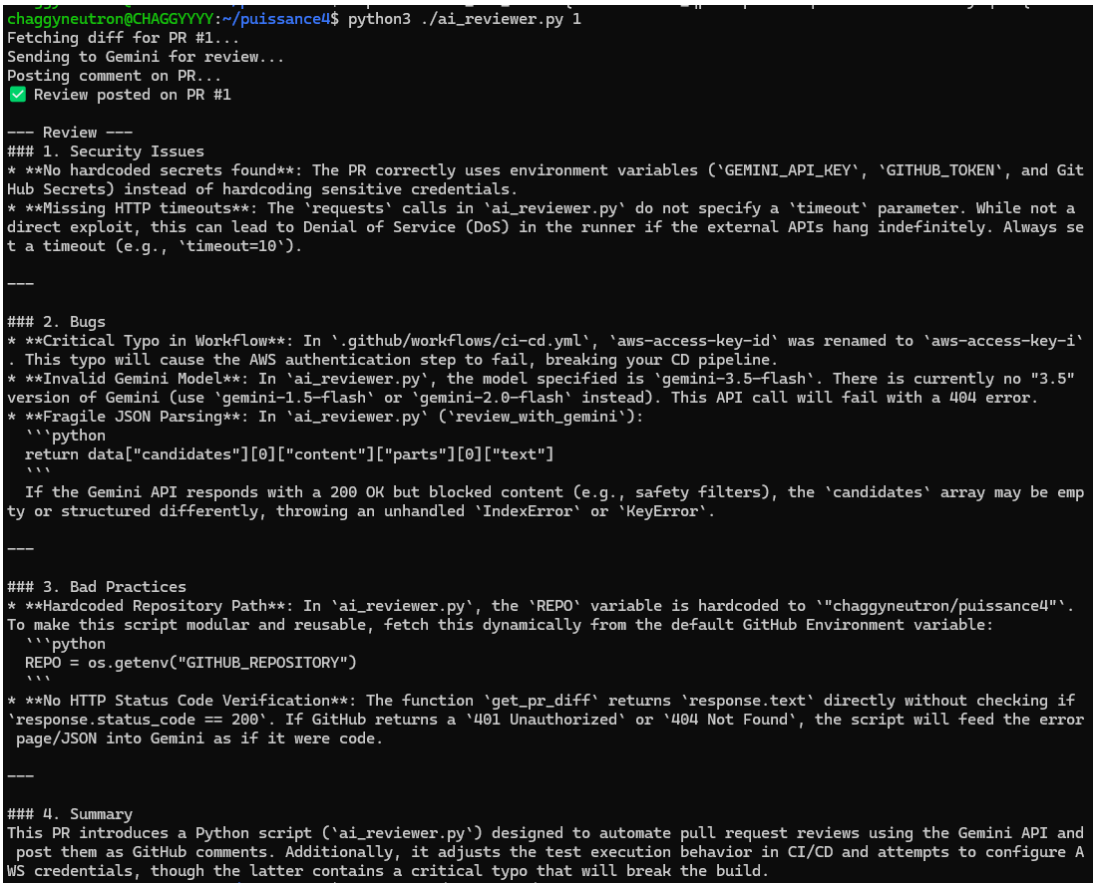


Figure 5: AI-generated review posted as a GitHub PR comment

Example Output

CI/CD Integration

The reviewer can be integrated as a GitHub Actions job that triggers automatically on every pull request:

```yaml
ai-review:
  runs-on: ubuntu-latest
  if: github.event_name == 'pull_request'
  steps:
    - uses: actions/checkout@v4
    - name: Run AI reviewer
      env:
        GEMINI_API_KEY: ${{ secrets.GEMINI_API_KEY }}
        GITHUB_TOKEN:   ${{ secrets.GITHUB_TOKEN }}
      run: python3 ai_reviewer.py ${{ github.event.pull_request
          .number }}
```

Listing 7: GitHub Actions job for automatic AI review

Component	Technology	Purpose
Backend API	Spring Boot 3, Java 17	REST + WebSocket
Database	PostgreSQL 16	Persistence
Authentication	JWT	Stateless auth
Containerization	Docker (multi-stage)	Portable deployment
Local orchestration	minikube + kubectl	Kubernetes dev environment
Production orchestration	AWS EKS + HPA	Auto-scaling deployment
CI/CD	GitHub Actions	Automated test, build, deploy
AI Reviewer	Python + Gemini 3.5 Flash	Automated PR code review
Frontend	HTML/CSS/JS	Single-file UI

Table 1: Full technology stack

Summary

The project demonstrates the full DevOps lifecycle — from local development with Docker Compose, through containerization and Kubernetes orchestration, to a complete CI/CD pipeline with an AI-augmented code review step integrated directly into the pull request workflow.